

```

#include<windows.h>
#include<GL/glut.h>
#include<iostream>
#include<cmath>
using namespace std;

float x1,x2,x3;
float Y_1,y2,y3;
float tx,ty;
float sx,sy;
float angle;
float shx,shy;
int x_axis,y_axis,origin;
void or_triangle(){
    glColor3f(0.0, 0.0, 0.0);
    glBegin(GL_LINES);
        glVertex2i(x1,Y_1);
        glVertex2i(x2,y2);
        glVertex2i(x2,y2);
        glVertex2i(x3,y3);
        glVertex2i(x3,y3);
        glVertex2i(x1,Y_1);
    glEnd();
}

void translation(){
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(3);
    or_triangle();
    glColor3f(0.0, 1.0, 0.0);
    glBegin(GL_LINES);
        glVertex2i(x1+tx,Y_1+ty);
        glVertex2i(x2+tx,y2+ty);
        glVertex2i(x2+tx,y2+ty);
        glVertex2i(x3+tx,y3+ty);
        glVertex2i(x3+tx,y3+ty);
        glVertex2i(x1+tx,Y_1+ty);
    glEnd();
    glFlush();
}

void scaling(){
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(3);
    or_triangle();
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_LINES);
        glVertex2i(x1*sx,Y_1*sy);
        glVertex2i(x2*sx,y2*sy);
        glVertex2i(x2*sx,y2*sy);
        glVertex2i(x3*sx,y3*sy);
        glVertex2i(x3*sx,y3*sy);
        glVertex2i(x1*sx,Y_1*sy);
    glEnd();
    glFlush();
}

void rotation(){
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(3);
    or_triangle();
    float rad = angle * 3.1416 / 180.0;

    float rx1 = x1*cos(rad) - Y_1*sin(rad);
    float rY_1 = x1*sin(rad) + Y_1*cos(rad);
    float rx2 = x2*cos(rad) - y2*sin(rad);
    float ry2 = x2*sin(rad) + y2*cos(rad);
    float rx3 = x3*cos(rad) - y3*sin(rad);
    float ry3 = x3*sin(rad) + y3*cos(rad);

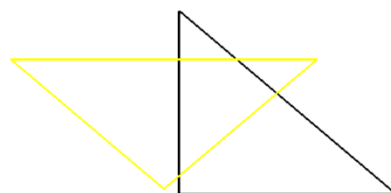
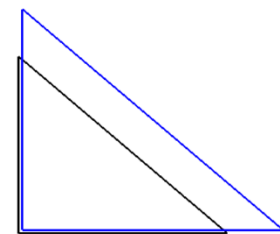
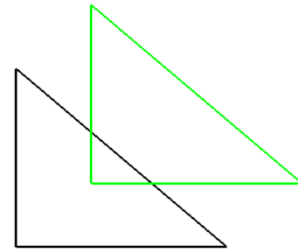
    glColor3f(1.0, 1.0, 0.0);
    glBegin(GL_LINES);
        glVertex2f(rx1,rY_1);
        glVertex2f(rx2,ry2);
        glVertex2f(rx2,ry2);
        glVertex2f(rx3,ry3);
        glVertex2f(rx3,ry3);
        glVertex2f(rx1,rY_1);
    glEnd();
}

```

```

E:\glutproject\060526\bin\De
Enter first point: 20 20
Enter second point: 300 20
Enter third point: 20 300
Enter translation factor: 100 100
Enter scaling factor: 1.25 1.25
Enter rotation angle (degree): 45
Enter shearing factors (shx shy): 0.2 0.3
reflection factor (x_axis,y_axis,origin)--(1 1 1):1 1 1

```



```

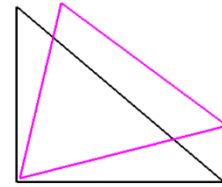
    glFlush();
}

void shearing(){
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(3);
    or_triangle();
    float shx1 = x1 + shx*y_1;
    float shY_1 = Y_1 + shy*x1;
    float shx2 = x2 + shx*y2;
    float shy2 = y2 + shy*x2;
    float shx3 = x3 + shx*y3;
    float shy3 = y3 + shy*x3;

    glColor3f(1.0, 0.0, 1.0);
    glBegin(GL_LINES);
        glVertex2f(shx1,shY_1);
        glVertex2f(shx2,shy2);
        glVertex2f(shx2,shy2);
        glVertex2f(shx3,shy3);
        glVertex2f(shx3,shy3);
        glVertex2f(shx1,shY_1);
    glEnd();
    glFlush();
}

```

shearing



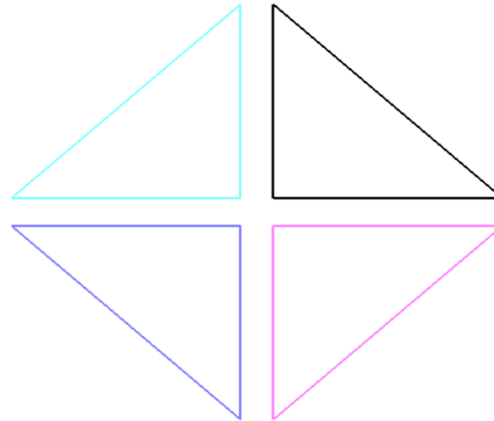
```

void reflection(){
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(3);
    or_triangle();

    if(x_axis){
        float refx1=x1, refY_1=Y_1;
        float refx2=x2, refy2=y2;
        float refx3=x3, refy3=y3;
        refY_1 = -Y_1;
        refy2 = -y2;
        refy3 = -y3;
        glColor3f(1.0, 0.5, 1.0);
        glBegin(GL_LINES);
            glVertex2f(refx1,refY_1);
            glVertex2f(refx2,refy2);
            glVertex2f(refx2,refy2);
            glVertex2f(refx3,refy3);
            glVertex2f(refx3,refy3);
            glVertex2f(refx1,refY_1);
        glEnd();
    }if(y_axis){
        float refx1=x1, refY_1=Y_1;
        float refx2=x2, refy2=y2;
        float refx3=x3, refy3=y3;
        refx1 = -x1;
        refx2 = -x2;
        refx3 = -x3;
        glColor3f(0.5, 1.0, 1.0);
        glBegin(GL_LINES);
            glVertex2f(refx1,refY_1);
            glVertex2f(refx2,refy2);
            glVertex2f(refx2,refy2);
            glVertex2f(refx3,refy3);
            glVertex2f(refx3,refy3);
            glVertex2f(refx1,refY_1);
        glEnd();
    }if(origin){
        float refx1=x1, refY_1=Y_1;
        float refx2=x2, refy2=y2;
        float refx3=x3, refy3=y3;
        refx1 = -x1; refY_1 = -Y_1;
        refx2 = -x2; refy2 = -y2;
        refx3 = -x3; refy3 = -y3;
        glColor3f(0.5, 0.5, 1.0);
        glBegin(GL_LINES);
            glVertex2f(refx1,refY_1);
            glVertex2f(refx2,refy2);
            glVertex2f(refx2,refy2);
            glVertex2f(refx3,refy3);
            glVertex2f(refx3,refy3);
            glVertex2f(refx1,refY_1);
        glEnd();
    }
}

```

reflection



```

    }

    glFlush();
}

void init()
{
    glClearColor(1,1,1,1);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(-500,500,-500,500);
}

int main(int argc, char *argv[])
{
    cout << "Enter first point: ";
    cin >> x1 >> y1;
    cout << "Enter second point: ";
    cin >> x2 >> y2;
    cout << "Enter third point: ";
    cin >> x3 >> y3;
    cout << "Enter translation factor: ";
    cin >> tx >> ty;
    cout << "Enter scaling factor: ";
    cin >> sx >> sy;
    cout << "Enter rotation angle (degree): ";
    cin >> angle;
    cout << "Enter shearing factors (shx shy): ";
    cin >> shx >> shy;
    cout << "reflection factor (x_axis,y_axis,origin)--(1 1 1):";
    cin >> x_axis >> y_axis >> origin;

    glutInit(&argc, argv);
    glutInitWindowSize(1000,1000);
    glutInitWindowPosition(0,0);
    glutInitDisplayMode(GLUT_RGB | GLUT_SINGLE);

    glutCreateWindow("translation");
    init();
    glutDisplayFunc(translation);

    glutCreateWindow("scaling");
    init();
    glutDisplayFunc(scaling);

    glutCreateWindow("rotation");
    init();
    glutDisplayFunc(rotation);

    glutCreateWindow("shearing");
    init();
    glutDisplayFunc(shearing);

    glutCreateWindow("refection");
    init();
    glutDisplayFunc(reflection);

    glutMainLoop();

    return 0;
}

```